PHP will also helpfully (or not so helpfully in some cases) *coerce* (automatically cast) data types for you when required. This can happen when certain operations require it; for example, a string might contain numerical characters, and if used in a mathematical operation (operations are discussed in the next section), it will be coerced to a numerical type (as we'll see in a later section of this chapter). PHP also has a type declaration system for functions. The inputs of functions can have their data types declared, and PHP will attempt to coerce incoming data at runtime if their types do not match. This is covered in depth in Chapter 3, "Functions."

The base data types used in PHP are as follows:

- `null`
- `bool` (boolean `true` or `false`)
- `integer` (a whole number, e.g., `123`)
- `float` (a number with a decimal point, e.g., `1.23`)
- `string` (a sequence of characters, e.g., `"Hello world"`)
- `array` (a collection of data)
- `object` (an instance of a class)
- `callable` (can be executed)
- `resource` (see below)

The first five types listed above are examples of *scalar* data. Scalars hold a single value: a boolean will be either true or false, an integer holds a single whole-number value, and so on. Strings, numbers, and booleans are commonly known as *primitive* types and are the building blocks of more complex data. Those of you with prior programming experience might be aware of IEEE (and other) standards concerning how computers store data, particularly numbers. PHP abstracts those types to the ones given above, but the underlying implementation does vary across different systems. This is covered in more detail in Chapter 7, "Numbers," and Chapter 8, "Strings."

Arrays and objects are *compound*, or *composite*, data types. They collect together multiple instances of other types – either scalars, other compound types, or a mixture of the two. (Unlike some strictly typed languages, PHP is quite happy to mix types in an array.)